VitrineFlex — Índice de ADRs

- **Baseline:** Documento Técnico v1.1
- **Data de aprovação:** 08/11/2026
- **Status:** ADR-001 a ADR-008 aceitas pelo responsável pelo projeto; nenhuma implementação iniciada por força destas decisões.

ADR	Título	Status
ADR-001	Escopo das features opcionais no MVP	Accepted
ADR-002	Roteamento determinístico e contexto da loja	Accepted
ADR-003	Baixa atômica de estoque e compensação idempotente	Accepted
ADR-004	Enforcement de features no backend com cache local	Accepted
ADR-005	SQLite por serviço com WAL e controle de contenção	Accepted
ADR-006	Imutabilidade do segmento e versionamento do manifesto	Accepted
ADR-007	Autenticação interna entre microsserviços	Accepted
ADR-008	Stack de bibliotecas do frontend	Accepted

Regra de governança

Uma ADR aceita não deve ser alterada silenciosamente. Uma mudança posterior deve gerar nova ADR que referencie e substitua a decisão anterior, mantendo os registros históricos. A documentação técnica deve refletir as decisões atualmente aceitas.

ADR-001 — Escopo das features opcionais no MVP

- **Projeto:** VitrineFlex
- **Data da decisão:** 08/11/2026
- **Status:** Accepted (aprovada pelo responsável pelo projeto)
- **Baseline:** Documento Técnico v1.1

1. Contexto

A baseline v1.0 listava Favoritos, Comparação de Produtos, Avaliações e Cupons como features opcionais, mas não especificava entidades, serviços e endpoints completos para todas. A inclusão simultânea das quatro funcionalidades aumentaria o escopo e criaria opções configuráveis sem comportamento backend correspondente.

2. Alternativas consideradas

- Alternativa A: implementar entidades e fluxos completos para todas as quatro features opcionais no MVP.
- **Alternativa B (escolhida):** incluir Favoritos e Comparação Tech; adiar Avaliações e Cupons.
- Alternativa C: eliminar todas as opcionais do MVP.

3. Decisão

O MVP implementará apenas:

- `favorites`: funcionalidade opcional aplicável aos três segmentos, com persistência no Catalog Service e integração com Recommendation Service.
- `product_comparison`: funcionalidade opcional exclusiva do segmento `TECH`, utilizando atributos técnicos do catálogo e composição frontend.

`reviews` e `coupons` ficam classificadas como extensões futuras. Não serão listadas como features selecionáveis, não estarão habilitadas em manifests do MVP e não terão endpoints funcionais nesta entrega. Uma requisição de configuração que contenha essas chaves deverá ser rejeitada como feature não suportada.

4. Justificativa

A seleção concentra o esforço em duas features que demonstram variabilidade de forma visível e testável, sem exigir um domínio adicional de moderação de avaliações ou cálculo/auditoria de cupons. Mantém a possibilidade de ampliar o modelo em versão posterior da LPS, sem prometer funcionalidades incompletas no MVP.

5. Consequências e trade-offs

- Favoritos acrescenta entidade ProductFavorite, rotas, validação de autenticação e evento para recomendação.
- Comparação exige atributos técnicos consistentes e só é renderizada em lojas TECH com a feature habilitada.
- Avaliações e cupons não podem ser anunciados como implementados na interface, README ou demonstração.
- A inclusão futura requer nova modelagem de dados, APIs, regras de negócio, testes e atualização formal do feature model.

6. Orientações de implementação

1. Manter o conjunto supported_features centralizado no LPS Configuration Service e documentado no feature model.
2. Modelar favorites como (store_id, user_id, product_id) no Catalog Service, com unicidade para evitar duplicação.
3. Registrar eventos FAVORITE e UNFAVORITE/remoção de favorito conforme contrato do Recommendation Service.
4. Permitir product_comparison apenas se segment == TECH.
5. O configurador não deve apresentar controles para reviews e coupons nesta versão.

7. Verificações e critérios de aceite

- Feature Favorites ativa: cliente autenticado consegue favoritar, listar e remover favoritos; dados não se misturam entre lojas.
- Feature Favorites inativa: endpoints retornam 403 FEATURE_NOT_ENABLED.
- Feature Comparison ativa em TECH: comparação de produtos é funcional.
- Tentativa de ativar Comparison em FASHION/BOOKS: configuração rejeitada com 422 INVALID_FEATURE_CONFIGURATION.
- Tentativa de enviar reviews ou coupons: configuração rejeitada como feature não suportada.
- Os três produtos demonstrativos exibem combinações consistentes com a regra.

8. Referências internas

- docs/VitrineFlex_Documento_Tecnico_v1.1.md
- docs/adr/ADR-INDEX.md

ADR-002 — Roteamento determinístico e contexto da loja

- **Projeto:** VitrineFlex
- **Data da decisão:** 08/11/2026
- **Status:** Accepted (aprovada pelo responsável pelo projeto)
- **Baseline:** Documento Técnico v1.1

1. Contexto

A baseline v1.0 misturava rotas por slug (/stores/{slug}/products) com rotas de serviço, forçando o Gateway e/ou o Catalog Service a resolver o slug em cada requisição de catálogo. Isso criava acoplamento e ambiguidade de roteamento.

2. Alternativas consideradas

- Alternativa A: Gateway resolve slug e reescreve todas as rotas.
- **Alternativa B (escolhida):** frontend carrega manifesto primeiro; Gateway usa prefixos fixos.
- Alternativa C: duplicar slug em cada serviço de domínio.

3. Decisão

A interface acessará /loja/{slug} e, na inicialização, consultará o LPS Configuration Service por meio de GET /api/v1/lps/stores/{slug}/manifest. O manifesto retornará store_id, segment, versão e features. Depois disso, o frontend enviará X-Store-ID em todas as chamadas de domínio.

O Gateway roteará exclusivamente por prefixos determinísticos:

- /api/v1/auth/* → Auth
- /api/v1/lps/* → LPS Configuration
- /api/v1/catalog/* → Catalog
- /api/v1/cart/* → Cart
- /api/v1/orders/* → Orders
- /api/v1/payments/* → Payments (quando houver rota pública; simulação de pagamento será interna no MVP)
- /api/v1/shipping/* → Shipping
- /api/v1/recommendations/* → Recommendation

4. Justificativa

A resolução slug→store_id ocorre uma vez no carregamento da loja. O Gateway permanece um proxy leve e previsível, e os serviços recebem contexto normalizado. O X-Store-ID é contexto solicitado, não prova de identidade nem de autorização. O Gateway verifica a presença/formato UUID, mas não consulta o LPS Configuration Service em toda requisição; cada serviço valida loja ativa/manifesto por meio do cache TTL e aplica escopo de dados/permissões do usuário.

5. Consequências e trade-offs

- Reduz chamadas repetidas para resolver slug e evita duplicar slug nos bancos de domínio.
- O frontend precisa tratar falha ao carregar o manifesto antes de liberar a vitrine.
- Um store_id alterado manualmente não pode permitir leitura ou escrita indevida; filtros tenant e autorização continuam obrigatórios.
- O Gateway se torna dependência de tráfego externo, mas não concentra regras de negócio.

6. Orientações de implementação

1. Publicar no LPS Config o endpoint GET /api/v1/lps/stores/{slug}/manifest.
2. Manter o slug como propriedade do LPS Configuration Service.
3. Após carregar o manifesto, armazenar o contexto da loja no estado da aplicação e enviar X-Store-ID.
4. O Gateway aplica tabelas explícitas de prefixo→serviço; não usa inspeção profunda de rota.
5. Criar rota interna GET /internal/v1/lps/stores/{store_id}/manifest no LPS Configuration para alimentar os caches locais dos microsserviços.
6. O Gateway remove os cabeçalhos X-User-ID, X-User-Role e X-Internal-Token fornecidos pelo cliente. Após validar JWT, injeta identidade confiável e injeta o segredo interno nas requisições encaminhadas à rede privada.
7. Todos os serviços de domínio validam X-Store-ID, existência/estado da loja via manifesto em cache, escopo de dados e autorização necessária.

7. Verificações e critérios de aceite

- Uma chamada ao manifesto por slug retorna loja ativa, store_id e manifesto.
- Requisições subsequentes usam prefixos determinísticos e o X-Store-ID correto.
- Cabeçalhos de identidade forjados pelo browser são removidos/ignorados.
- Cliente de uma loja não consegue acessar carrinho, pedidos ou favoritos de outra loja alterando X-Store-ID.
- Requisição sem contexto de loja em endpoint comercial retorna 400 STORE_CONTEXT_REQUIRED.
- O Gateway não efetua uma chamada síncrona ao LPS Config em cada requisição de catálogo/carrinho/pedidos.

8. Referências internas

- docs/VitrineFlex_Documento_Tecnico_v1.1.md
- docs/adr/ADR-INDEX.md

ADR-003 — Baixa atômica de estoque e compensação idempotente

- **Projeto:** VitrineFlex
- **Data da decisão:** 08/11/2026
- **Status:** Accepted (aprovada pelo responsável pelo projeto)
- **Baseline:** Documento Técnico v1.1

1. Contexto

O checkout inicial consultava disponibilidade antes do pagamento, mas não definia baixa atômica nem reversão. Duas compras simultâneas poderiam vender a última unidade mais de uma vez; uma recusa de pagamento também poderia deixar estoque inconsistente.

2. Alternativas consideradas

- Alternativa A (escolhida): baixa atômica antes do pagamento e compensação se pagamento falhar.
- Alternativa B: reserva temporária com TTL, commit e release, exigindo mecanismo de expiração.
- Alternativa C: baixar estoque somente após aprovação, aceitando risco de cancelamento por falta de saldo.

3. Decisão

No MVP, Orders orquestrará um fluxo síncrono com compensação:

1. Requer Idempotency-Key no checkout.
2. Revalida carrinho, preço, variantes e estoque.
3. Cria pedido PENDING_PAYMENT com snapshot e manifest_version.
4. Solicita ao Catalog a baixa atômica de estoque antes da simulação de pagamento.
5. Catalog realiza update condicional (available_stock >= requested_quantity) dentro de transação e registra uma movimentação idempotente.
6. Payments simula a transação com sua própria chave idempotente.
7. Em sucesso, Orders marca PAID e limpa carrinho.
8. Em falha, Orders solicita restauração idempotente de estoque e marca PAYMENT_FAILED.
9. Se a compensação não concluir, o pedido permanece COMPENSATION_PENDING e pode ser reconciliado/repetido por operação administrativa.
10. Se houver timeout/resposta ambígua no Payments, Orders mantém PAYMENT_PENDING_CONFIRMATION e consulta/reexecuta o mesmo Idempotency-Key; não libera estoque até conhecer resultado definitivo.

4. Justificativa

A baixa antes do pagamento simulado evita que duas compras confirmem o mesmo saldo. A compensação síncrona é mais simples para o MVP do que uma reserva com TTL e worker de expiração. A idempotência protege contra retries de rede e repetição pelo usuário. O padrão é uma Saga orquestrada simples, sem afirmar transação distribuída ACID.

5. Consequências e trade-offs

- Exige estado adicional de pedido e registro de movimentações de estoque.
- Uma falha prolongada na compensação precisa ser visível e reconciliável; não será escondida como sucesso.
- Timeout ambíguo de pagamento não pode ser tratado como recusa definitiva: manter pedido pendente de confirmação e consultar com a mesma chave idempotente antes de liberar estoque.
- O fluxo pode manter estoque reduzido enquanto pagamento simulado está em andamento.
- Não há job automático de TTL; a reconciliação será via endpoint/ação administrativa no MVP.

6. Orientações de implementação

1. Orders gera/recebe Idempotency-Key única por cliente e loja, persiste a chave com unicidade.
2. Catalog expõe apenas internamente POST /internal/v1/inventory/decrement e POST /internal/v1/inventory/restore.
3. As operações internas recebem operation_id estável e registram InventoryMovement para impedir duplicação.
4. O decremento usa transação curta e operação condicional; estoque insuficiente retorna conflito de domínio.
5. Payments persiste PaymentAttempt com referência de idempotência.
6. Orders registra transições de estado e erro de compensação.
7. Não limpar o carrinho antes de PAID.
8. Definir estado PAYMENT_PENDING_CONFIRMATION para resposta ambígua/timeout; só executar a compensação após resultado definitivo de falha.

7. Verificações e critérios de aceite

- Duas requisições concorrentes para a última unidade: no máximo uma baixa é aprovada.
- Repetir checkout com a mesma chave retorna o pedido anterior sem gerar novo pedido ou nova baixa.
- Pagamento recusado restaura estoque uma única vez.

- Repetir restauração não incrementa estoque novamente.
- Falha da restauração mantém COMPENSATION_PENDING com registro de erro.
- Timeout de Payments não libera estoque; consulta/retry com a mesma chave resolve o mesmo resultado sem nova transação.
- Pedido pago preserva snapshots e manifesto usados no instante da compra.

8. Referências internas

- docs/VitrineFlex_Documento_Tecnico_v1.1.md
- docs/adr/ADR-INDEX.md

ADR-004 — Enforcement de features no backend com cache local

- **Projeto:** VitrineFlex
- **Data da decisão:** 08/11/2026
- **Status:** Accepted (aprovada pelo responsável pelo projeto)
- **Baseline:** Documento Técnico v1.1

1. Contexto

A LPS exige que uma funcionalidade desabilitada não possa ser executada por chamada direta à API. Consultar o LPS Configuration Service a cada operação sensível introduziria latência e forte acoplamento entre serviços.

2. Alternativas consideradas

- Alternativa A: chamada HTTP ao LPS a cada requisição.
- Alternativa B: Gateway injeta lista de features em cabeçalho assinado para todos os serviços.
- **Alternativa C (escolhida):** cache local por microsserviço, encapsulado em python-common, TTL 30s.

3. Decisão

Cada microsserviço que executa operação condicionada por feature utilizará um verificador comum do pacote python-common. O verificador buscará o ProductManifest no LPS Configuration Service e manterá cache local em memória com **TTL de 30 segundos** por store_id/versão. Durante a validade, as operações reutilizam o manifesto em cache. Ao expirar, o serviço atualiza a entrada.

Se a feature necessária estiver desligada, a operação será bloqueada com HTTP 403 e código FEATURE_NOT_ENABLED. Uma combinação de configuração inválida retorna 422 INVALID_FEATURE_CONFIGURATION.

4. Justificativa

A opção evita chamada de rede em cada requisição e deixa o enforcement próximo do domínio. O TTL de 30 segundos limita a janela de eventual consistência quando a configuração muda, sem adicionar mensageria para invalidação. A cache não substitui verificação de identidade, autorização nem isolamento por loja.

5. Consequências e trade-offs

- Uma mudança de feature poderá levar até 30 segundos para refletir em um microsserviço que possua cache vigente.
- A cache precisa ser indexada por store_id e deve considerar manifest_version.
- Se o LPS Config estiver indisponível e não houver manifesto válido em cache, a operação protegida deve falhar de forma segura (fail closed), sem assumir a feature ativa.
- O pacote compartilhado deve permanecer pequeno e técnico, sem conter regras de domínio de negócio.

6. Orientações de implementação

1. Implementar FeatureManifestProvider em packages/python-common.
2. Chavear cache por store_id; armazenar timestamp de expiração e manifest_version.
3. Usar TTL padrão de 30 segundos configurável por ambiente para testes, mantendo 30s como baseline.
4. Exigir chamada de verificação nas rotas associadas a Favoritos e Comparação, e em qualquer outra operação opcional futura.
5. Retornar 403 FEATURE_NOT_ENABLED se a feature estiver falsa/ausente.
6. Validar também o segmento para features segment-specific.

7. Em falha de obtenção sem cache válida, bloquear a operação e registrar erro com X-Request-ID.

7. Verificações e critérios de aceite

- Feature ativa na loja: endpoint retorna comportamento esperado.
- Feature inativa: chamada direta retorna 403 FEATURE_NOT_ENABLED.
- Feature comparação em segmento não Tech: retorna 422 INVALID_FEATURE_CONFIGURATION ou regra de feature incompatível, conforme contexto.
- Cache reutiliza manifesto dentro de 30s e busca nova versão após expiração.
- Cache sem manifesto e serviço LPS indisponível: falha fechada; não libera recurso por padrão.

8. Referências internas

- docs/VitrineFlex_Documento_Tecnico_v1.1.md
- docs/adr/ADR-INDEX.md

ADR-005 — SQLite por serviço com WAL e controle de contenção

- **Projeto:** VitrineFlex
- **Data da decisão:** 08/11/2026
- **Status:** Accepted (aprovada pelo responsável pelo projeto)
- **Baseline:** Documento Técnico v1.1

1. Contexto

A disciplina sugere Python e os trabalhos de referência utilizaram Docker e SQLite. A baseline propõe database-per-service. Porém, o SQLite possui limitações de concorrência, especialmente em múltiplas escritas simultâneas.

2. Alternativas consideradas

- Alternativa A: PostgreSQL central com bancos/schemas por serviço.
- **Alternativa B (escolhida):** SQLite por serviço com WAL, timeout e tratamento de contenção.

3. Decisão

Manter um arquivo SQLite independente por serviço, com SQLAlchemy, e padronizar:

- PRAGMA journal_mode=WAL;
- PRAGMA busy_timeout=5000 (5 segundos);
- transações curtas;
- índices para store_id e filtros frequentes;
- sessão/conexão isolada por unidade de trabalho;
- retry limitado somente para operações que sejam seguras/idempotentes;
- volumes Docker próprios para persistência de cada serviço.

A documentação afirmará explicitamente que WAL permite melhor coexistência de leitores e escritor, mas continua existindo apenas um escritor ativo por vez.

4. Justificativa

A decisão respeita a simplicidade do ambiente acadêmico, elimina necessidade de serviço de banco externo e mantém propriedade dos dados por microsserviço. As configurações reduzem bloqueios transitórios, mas não transformam SQLite em banco de escrita altamente concorrente.

5. Consequências e trade-offs

- Menor esforço de instalação e execução local.
- Cada serviço controla seu próprio esquema e arquivo.
- Cargas elevadas de escrita podem serializar e elevar latência.
- Os testes devem incluir contenção e checkout concorrente.
- Migração futura para PostgreSQL por serviço permanece possível, mas fora do MVP.

6. Orientações de implementação

1. Configurar WAL e busy_timeout na inicialização de cada engine/conexão.
2. Não compartilhar arquivo .db entre microsserviços.
3. Evitar transações abertas durante chamadas HTTP externas.
4. Executar chamadas de rede fora da transação de escrita sempre que possível.
5. Aplicar update condicional de estoque numa transação curta.
6. Documentar volumes e backups locais; não incluir bancos de demonstração com dados pessoais reais no Git.

7. Verificações e critérios de aceite

- Cada serviço cria/abre apenas seu banco.
- PRAGMA journal_mode retorna wal.
- PRAGMA busy_timeout retorna 5000.
- Teste com leituras concorrentes e tentativas simultâneas de estoque não corrompe dados.
- Teste demonstra que, sob escrita concorrente, há serialização/espera e não se promete múltiplos escritores simultâneos.

8. Referências internas

- docs/VitrineFlex_Documento_Tecnico_v1.1.md
- docs/adr/ADR-INDEX.md

ADR-006 — Imutabilidade do segmento e versionamento do manifesto

- **Projeto:** VitrineFlex
- **Data da decisão:** 08/11/2026
- **Status:** Accepted (aprovada pelo responsável pelo projeto)
- **Baseline:** Documento Técnico v1.1

1. Contexto

A configuração de uma loja pode mudar após haver produtos, favoritos e pedidos. Alterar o segmento comercial após a persistência do catálogo poderia invalidar atributos, variantes e histórico. Desligar uma feature com dados existentes também pode afetar a consulta histórica.

2. Alternativas consideradas

- Alternativa A: manifesto totalmente mutável, sem histórico.
- **Alternativa B (escolhida):** segmento imutável e opcionais versionadas.
- Alternativa C: manifesto completamente imutável; qualquer alteração exigiria nova loja.

3. Decisão

O segmento (FASHION, TECH ou BOOKS) será imutável após a criação da loja. Alterações permitidas em features comportamentais/opcionais gerarão uma nova revisão do ProductManifest (manifest_version = manifest_version + 1). O manifesto anterior será mantido para rastreabilidade.

Orders armazenará a manifest_version utilizada na compra. Desabilitar feature não apagará os dados históricos: os registros serão preservados, mas a interface e as operações novas seguirão a versão atual. Features estruturais ou com dados incompatíveis não poderão ser desativadas até que exista estratégia explícita e segura.

4. Justificativa

A regra separa variabilidade estrutural do segmento e variabilidade comportamental. Evita que uma loja de livros seja convertida em loja de moda por atualização de configuração e preserva evidência de qual configuração gerou cada pedido.

5. Consequências e trade-offs

- A configuração deixa de ser sobrescrita sem histórico.

- A recuperação da loja sempre usa a versão corrente, enquanto pedidos preservam a versão transacional.
- Features podem ser ocultadas/desativadas sem apagar registros históricos.
- A validação pode rejeitar atualização que afete dados em uso; derivar outra loja pode ser necessário para alterações estruturais.

6. Orientações de implementação

1. ProductManifest deve possuir manifest_version, schema_version, created_at e referência à loja.
2. O LPS Configuration Service mantém histórico ordenado das versões.
3. segment não é editável após a criação.
4. Alteração de opcionais cria versão nova, sem atualizar o registro antigo in place.
5. Orders grava a versão vigente no momento de checkout.
6. A regra de ativação de feature deve ser aplicada com a versão atual, respeitando o TTL de cache definido em ADR-004.

7. Verificações e critérios de aceite

- Manifesto v1 continua consultável após criação da v2.
- Segmento não pode ser alterado por PATCH.
- Feature Comparison não pode ser habilitada em loja não Tech.
- Pedidos antigos continuam exibindo snapshots mesmo após desativação de feature.
- Feature com dados incompatíveis não pode ser desativada sem regra de migração/compatibilidade.

8. Referências internas

- docs/VitrineFlex_Documento_Tecnico_v1.1.md
- docs/adr/ADR-INDEX.md

ADR-007 — Autenticação interna entre microsserviços

- **Projeto:** VitrineFlex
- **Data da decisão:** 08/11/2026
- **Status:** Accepted (aprovada pelo responsável pelo projeto)
- **Baseline:** Documento Técnico v1.1

1. Contexto

Os serviços precisam executar chamadas internas como baixa de estoque, restauração, simulação de pagamento e consulta de dados. Essas rotas não devem ser acessíveis livremente pelo navegador nem aceitar cabeçalhos de contexto forjados.

2. Alternativas consideradas

- Alternativa A: mTLS entre containers.
- **Alternativa B (escolhida):** X-Internal-Token compartilhado em rede Docker privada, com contexto validado.
- Alternativa C: apenas repassar JWT do usuário para todos os fluxos internos.

3. Decisão

As rotas internas utilizarão X-Internal-Token, cujo valor será fornecido por variável de ambiente (INTERNAL_SERVICE_KEY) e compartilhado apenas entre containers autorizados na rede privada Docker. Os endpoints internos utilizarão prefixo /internal/v1/* e não serão publicados no roteamento público do Gateway.

O contexto de usuário/loja será propagado separadamente por X-User-ID, X-User-Role e X-Store-ID, mas esses cabeçalhos só serão confiáveis quando originados após validação no Gateway ou em chamada interna autenticada. O Gateway deve remover os mesmos cabeçalhos se enviados diretamente pelo navegador; em seguida injeta o segredo interno nas chamadas encaminhadas aos serviços na rede privada. O X-Store-ID é validado quanto ao formato na borda e validado quanto à existência/estado pelo serviço de domínio via manifesto em cache.

4. Justificativa

Um segredo compartilhado é mais simples de configurar, demonstrar e testar no escopo local do que mTLS, e impede chamadas internas triviais fora do ambiente autorizado. A rede privada Docker e a não publicação de endpoints internos reduzem exposição. Não equivale a identidade criptográfica individual por serviço e não deve ser apresentado como solução de produção.

5. Consequências e trade-offs

- Um segredo comum precisa ser protegido; vazamento afeta todos os serviços que o aceitam.
- Rotação de segredo exige atualizar ambiente/containers.
- A rede Docker privada e bind de portas em loopback são parte da decisão.
- JWT do usuário continua necessário para autorização de ações em nome do cliente; token interno não substitui autorização de domínio.

6. Orientações de implementação

1. Guardar INTERNAL_SERVICE_KEY em .env local ignorado pelo Git e documentar apenas a variável em .env.example.
2. Validar X-Internal-Token em toda rota /internal/v1/*.
3. Não mapear rotas internas no Gateway para o browser.
4. Publicar portas locais de microsserviços apenas em 127.0.0.1 para a demonstração acadêmica; em implantação pública manter serviços não expostos.
5. O Gateway remove cabeçalhos de identidade/segredo recebidos do cliente, valida JWT, injeta identidade confiável e inclui o segredo interno ao encaminhar a requisição à rede privada.
6. O Gateway valida presença/formato de X-Store-ID; o microsserviço valida loja ativa por manifesto/cache e aplica papel, membership e propriedade do recurso. O segredo interno isoladamente não concede permissão comercial.

7. Verificações e critérios de aceite

- Chamada interna sem token ou token inválido retorna 401/403.
- Chamada interna com token válido, identidade e escopo corretos é aceita.
- Browser não consegue chamar /internal/v1/* pelo Gateway.
- Cabeçalhos X-User-ID/X-User-Role forjados não alteram a identidade autenticada.
- .env real está no .gitignore; segredo não aparece em logs ou repositório.

8. Referências internas

- docs/VitrineFlex_Documento_Tecnico_v1.1.md
- docs/adr/ADR-INDEX.md

ADR-008 — Stack de bibliotecas do frontend

- **Projeto:** VitrineFlex
- **Data da decisão:** 08/11/2026
- **Status:** Accepted (aprovada pelo responsável pelo projeto)
- **Baseline:** Documento Técnico v1.1

1. Contexto

A baseline definia React, TypeScript e Vite, mas deixava abertas as escolhas de estilo, navegação, cache de requisições e ícones. O projeto precisa de temas por loja derivada e componentes reutilizáveis com estados previsíveis de carregamento/erro.

2. Alternativas consideradas

- **Alternativa A (escolhida):** Tailwind CSS + Lucide React + TanStack Query + React Router.
- Alternativa B: CSS Modules puros e Fetch nativo.

3. Decisão

A stack do frontend será:

- React + TypeScript;
- Vite;
- Tailwind CSS;
- React Router;
- TanStack Query;
- Lucide React.

As versões exatas serão fixadas durante a preparação do ambiente após verificação de compatibilidade. O manifesto do produto fornecerá a identidade/tema da loja; variáveis CSS e classes Tailwind permitirão sua aplicação sem duplicar layouts por segmento.

4. Justificativa

A combinação atende à composição modular, rotas `/loja/{slug}` e `/admin`, cache e sincronização de APIs, design responsivo e identidade visual configurável. Reduz decisões abertas sem exigir uma biblioteca complexa de estado global.

5. Consequências e trade-offs

- Introduz dependências adicionais a serem fixadas e auditadas.
- TanStack Query cuidará de dados remotos; estado local de formulário/visual será mantido no React.
- Tailwind requer convenção consistente de componentes e tokens de tema.
- A stack não substitui a arquitetura de componentes de domínio nem a validação de features no backend.

6. Orientações de implementação

1. Organizar componentes comuns em `apps/web/src/components`.
2. Organizar componentes de segmento em `apps/web/src/segments/fashion`, `tech` e `books`.
3. Organizar módulos opcionais em `apps/web/src/features/favorites` e `product-comparison`.
4. Carregar manifesto e inicializar um `StoreContext` antes de renderizar rotas da loja.
5. Aplicar rotas/slots por manifesto; comparação apenas se `segment == TECH` e `product_comparison == true`.
6. Utilizar TanStack Query para chamadas e cache remoto; React Router para rotas; Tailwind para tema/estilo; Lucide React para ícones.

7. Verificações e critérios de aceite

- As três lojas usam os mesmos componentes-base (ProductCard, ProductGrid, CartDrawer, CheckoutStepper etc.).
- Segmentos renderizam somente seu painel de atributos.
- Features opcionais controlam rotas/slots no frontend e não substituem enforcement no backend.
- Temas distintos são aplicados usando dados do manifesto sem duplicar a aplicação.
- Estados de loading, erro e cache de chamadas são tratados com TanStack Query.

8. Referências internas

- `docs/VitrineFlex_Documento_Tecnico_v1.1.md`
- `docs/adr/ADR-INDEX.md`